

# Reinforcement Learning vs. Rule-Based Policies for Dynamic Credit Threshold Control: A Simulation Study

Evgeny Baulin<sup>a,1,\*</sup>, P. P. Lukianchenko<sup>b,2</sup>

<sup>a</sup>*HSE University, Faculty of Computer Science, Bachelor's Programme "Data Science and Business Analytics", Moscow, Russia*

<sup>b</sup>*Trusted AI Research Center, Ivannikov Institute for System Programming of the RAS (ISP RAS), Moscow, Russia*

---

## Abstract

Portfolio-level credit acceptance thresholds are revised periodically in practice, which makes the decision sequential rather than one-shot. Each week the controller sets two acceptance thresholds, one for new clients and one for repeat clients, and then observes delayed repayment and default outcomes over subsequent weeks. We evaluate six deep reinforcement learning (RL) algorithms (DQN, Double-DQN, PPO, SAC, A3C, and A2C) against five rule-based baselines in a calibrated synthetic credit-portfolio simulator across six market scenarios and four observation-space sizes (12D, 20D, 30D, 50D). Data-driven weekly threshold control is one concrete way intelligent systems can improve business lending decisions, because adjusting thresholds to current portfolio and market signals can reduce realized default rates while protecting

---

\*Corresponding author.

*Email addresses:* `eabaulin@edu.hse.ru` (Evgeny Baulin),  
`p.lukyanchenko@ispras.ru` (P. P. Lukianchenko)

<sup>1</sup>ORCID: 0009-0004-9030-2893.

<sup>2</sup>ORCID: 0000-0002-1022-6041.

profit, and this study identifies when such automation pays off. The simulator is a calibrated proof-of-concept test bench, not a substitute for real banking data. Only observation dimensionality varies across the four conditions, with the simulator, reward, seeds, and evaluation protocol held fixed. We find that the constraint-aware weekly policy is the best overall controller by cumulative reward (the primary ranking metric, with realized profit 3,802,547) and is statistically tied with the profit-oriented policy on realized profit, and no RL agent wins any scenario by reward or profit. Double-DQN is the strongest RL agent, peaking at 12D (realized profit 3,446,564) and declining at higher state dimensionality. Richer-state RL agents nonetheless achieve lower portfolio default rates than the two profit-seeking baselines in a majority of scenarios, while the risk-aware weekly policy remains the lowest-default controller overall, a conditional risk-side advantage. A further central finding is that the best RL policies are episode-static, fixing a threshold pair at the start of each episode and barely adjusting it within the horizon, so richer observations improve initial threshold selection rather than genuine week-to-week adaptation under the short training budget. We conclude that well-designed rule-based weekly threshold search remains the benchmark when training budgets are short.

*Keywords:* Reinforcement Learning, Credit Scoring, Threshold Control, Markov Decision Process, Delayed Rewards, Portfolio Simulation, Rule-Based Policies, State Dimensionality

---

## 1. Introduction

A credit manager who holds a single static acceptance threshold loses money as the portfolio composition, the capital usage, and the market con-

ditions drift between decisions. In practice, banks and consumer-lending institutions revise acceptance thresholds periodically rather than once. The fraction of applicants to approve this week depends on the current portfolio composition, outstanding capital commitments, rolling default rates, and market conditions that evolve between decisions. The weekly acceptance decision is therefore a recurring control problem, not a one-shot cut-off. Much of the academic literature has instead treated threshold selection as a one-shot classification problem, estimating a default probability, choosing a cut-off, and evaluating by AUC or Gini, which ignores the sequential nature of actual threshold management (Thomas, 2017; Lessmann et al., 2015). That gap between static cut-off optimisation and dynamic portfolio management motivates the present study.

We build and evaluate an intelligent decision-support module for weekly threshold control, and we compare learned (RL) controllers against strong rule-based expert policies. Data-driven weekly threshold control is one concrete way artificial intelligence can improve business lending decisions. By adjusting acceptance thresholds to current portfolio and market signals, it can reduce realized defaults while protecting profit, and the present study identifies the conditions under which such automation pays off. Reinforcement learning provides a natural framework for decisions with delayed consequences (Arulkumaran et al., 2017). An RL agent observes the state of the portfolio, selects a pair of weekly thresholds, and receives a reward signal that reflects realized portfolio cash flows rather than immediate cohort expectations. The delayed-outcome setting is central: a loan accepted at week  $t$  may not default or repay until week  $t + \Delta$ , so the correct evaluation object is the full portfolio

trajectory, not the immediately observable cohort. Herasymovych et al. (2019) showed that a DQN agent can learn to adjust a single acceptance threshold in a simulated credit environment with delayed outcomes, establishing the structural ideas on which this study builds.

*Calibration disclaimer.* All experiments in this paper use a synthetic simulator calibrated to published retail lending statistics (Bank of Russia supervisory data and ECB SSM loan loss reports) as a controlled proof-of-concept test bench. Synthetic simulation is standard practice in RL-based credit research because real loan-level panels with weekly cohort granularity and observable repayment and default events are not publicly available (Herasymovych et al., 2019; Lessmann et al., 2015). The simulator provides full control over market regime, segment composition, and shock timing, which makes the multi-scenario robustness evaluation possible. Results should be interpreted as evidence about algorithmic behaviour under these controlled conditions, not as forecasts for any specific production portfolio.

The paper extends the framework of Herasymovych et al. (2019) with four contributions: (i) split weekly thresholds for new and repeat client segments, (ii) a controlled state-dimensionality experiment comparing 12D, 20D, 30D, and 50D observation sizes under a fixed protocol, (iii) five economically meaningful rule-based baselines as the comparison standard, and (iv) statistical hypothesis tests on paired run-level data. The central research question is:

*Under which conditions does reinforcement learning provide a robust improvement over rule-based threshold policies in a delayed-outcome credit-scoring environment?*

The data give a conditional answer. Strong rule-based weekly search wins every scenario and dimension by headline metrics, but richer-state RL agents produce lower portfolio default rates than the profit-seeking baselines in a majority of scenarios, while the risk-aware weekly policy stays the lowest-default controller overall. This points to a conditional risk-side advantage that becomes visible once the profit ranking is supplemented by credit-quality metrics.

## **2. Related Work**

Traditional credit scoring estimates default probability at the application level and selects a cut-off that balances interest income against expected loss (Thomas, 2017; Hand & Henley, 1997; Siddiqi, 2006). Benchmarking studies have shown that classifier rankings depend on the dataset and the evaluation protocol, and that logistic regression and gradient-boosted trees are difficult to outperform with more complex classifiers (Lessmann et al., 2015; Baesens et al., 2003). The same conclusion recurs in more recent work: Gunnarsson et al. (2021) find that deep neural networks do not reliably beat gradient boosting on standard credit datasets, and reviews of consumer credit risk assessment reach a similar verdict on where modelling effort actually pays off (Zhang & Yu, 2023). Application-oriented research has therefore shifted from raw classifier accuracy toward feature enhancement and ensembling (Yang et al., 2024) and toward explainability for credit decisions (Nallakaruppan et al., 2024), reflecting the practical priorities of intelligent lending systems.

These advances still treat the accept/reject cut-off as a static one-shot tuning step. A second line of work reframes threshold choice as a cost-sensitive

economic decision rather than a classification metric (Baesens et al., 2003), and Bellotti & Crook (2009) note that under Basel II/III capital adequacy requirements the threshold affects outstanding exposure, realized losses, and regulatory capital in ways that make static cut-off selection insufficient once market conditions or portfolio composition drift over time. Once the decision is recognised as recurring and state-dependent, it becomes a sequential control problem, which is the natural domain of reinforcement learning.

Reinforcement learning addresses problems with delayed consequences through the Markov Decision Process (MDP) framework (Dayan & Niv, 2008; Arulkumaran et al., 2017). The deep RL algorithms used in this study illustrate the main design axes. DQN couples Q-learning with experience replay for off-policy sample efficiency on discrete action spaces, Double-DQN reduces the overestimation bias of vanilla DQN (van Hasselt et al., 2016), PPO is an on-policy policy-gradient method with stable updates (Schulman et al., 2017), SAC is an off-policy maximum-entropy actor-critic (Haarnoja et al., 2018), and A3C and A2C are asynchronous and synchronous actor-critic variants. A broad survey of these methods appears in Arulkumaran et al. (2017). In this study A2C, PPO, and SAC use the Stable-Baselines3 implementations (Raffin et al., 2021), while DQN, Double-DQN, and A3C use matched custom PyTorch implementations.

Applying RL to credit control is more recent and still thin. Herasymovych et al. (2019) optimise the acceptance threshold of a credit-scoring model with RL in a simulated environment with delayed outcomes, and Kiatsupaibul et al. (2024) survey reinforcement learning for credit scoring and underwriting and note how few controlled comparisons against strong rule-based policies

exist. The direct precursor (Herasymovych et al., 2019) compared a DQN agent against a single static threshold in one scenario using a shared, non-split threshold. No prior work has simultaneously evaluated split thresholds for distinct borrower segments, a controlled observation-dimensionality comparison, strong economically motivated rule-based baselines, and six market scenarios with statistical testing. That combination is the gap this paper fills, and it motivates the three hypotheses stated in Section 4.1.

### 3. Problem Formulation and Methodology

#### 3.1. MDP Formulation

We model the weekly threshold-control problem as a discrete-time MDP  $(\mathcal{S}, \mathcal{A}, \mathcal{P}, r, \gamma)$ . The **state**  $s_t \in \mathbb{R}^d$  ( $d \in \{12, 20, 30, 50\}$ ) is a portfolio summary vector observed at the beginning of interactive week  $t$ . It includes approval rates, default rates, realized profit and NPV, capital usage, outstanding exposure, segment shares, and, at higher dimensionalities, lagged and rolling history. The **action** is a pair of acceptance thresholds  $a_t = (\tau_t^{\text{new}}, \tau_t^{\text{repeat}}) \in \{35, 40, \dots, 85\}^2$ : applications from new (repeat) clients with credit score above  $\tau_t^{\text{new}}$  ( $\tau_t^{\text{repeat}}$ ) are accepted. This yields  $11 \times 11 = 121$  joint actions per week. The **reward** is described in Section 3.4. The discount factor is  $\gamma = 0.98$ .

Figure 1 shows the weekly decision loop as a Markov decision process. The figure makes explicit that each weekly action is scored only through delayed cash flows, so the controller must account for outcomes that arrive several weeks after the threshold is set.

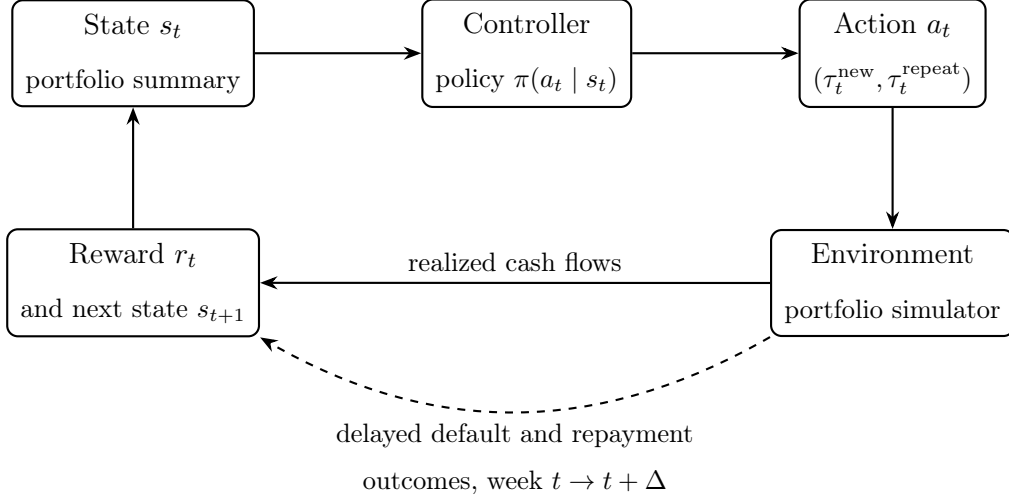


Figure 1: Weekly threshold control as a Markov decision process. The dashed arrow marks the delayed default and repayment outcomes that link an action taken at week  $t$  to rewards realized at later weeks  $t + \Delta$ .

### 3.2. Simulation Environment

Each episode consists of three phases. (1) An **8-week warm-up** pre-fills the loan book using default thresholds before the controller begins acting. (2) During the **52-week interactive horizon** the controller sets weekly thresholds. The simulator generates applications from Gaussian credit scores clipped to  $[0, 100]$  for new and repeat segments, applies a logistic default-probability mapping, accepts loans above the threshold, and schedules repayment, default, and recovery events in a forward event queue. (3) **Terminal settlement** resolves all pending cash flows at episode end so that loans originated near the horizon are not mechanically penalized. Default probabilities are calibrated to target 4–6% in the base scenario and 9–14% under adverse stress, consistent with published retail lending statistics.

Figure 2 shows the three-phase episode structure and the within-week loan



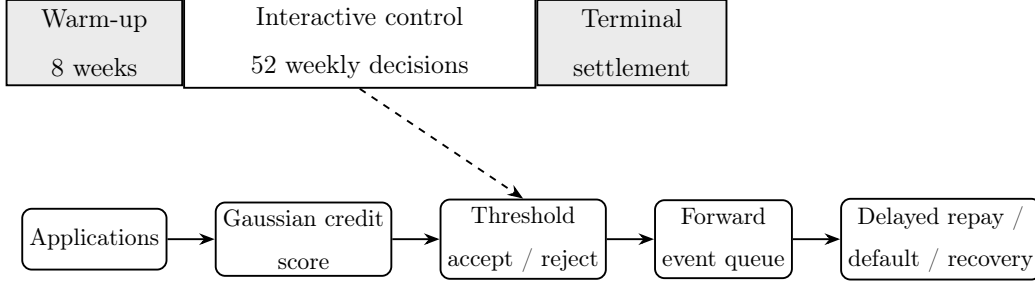


Figure 2: Episode structure and loan lifecycle. The top row is the episode timeline (8-week warm-up, 52-week interactive weekly control, terminal settlement), and the bottom row is the within-week sequence from application to delayed outcome.

lifecycle. The figure shows that outcomes from accepted loans are pushed into a forward event queue, so repayments, defaults, and recoveries feed back into later weeks rather than into the week of origination.

In Table 1, the six evaluation scenarios and their primary stress mechanism are listed. The scenarios probe distinct portfolio-management weaknesses, so a controller that performs well across all of them is robust rather than tuned to a single regime.

### 3.3. Agents and Baselines

**RL agents.** All six agents use a shared two-layer MLP ( $2 \times 128$  hidden units) and a fixed set of hyperparameters held identical across the four state sizes (Appendix B). A2C, PPO, and SAC use Stable-Baselines3, while DQN, Double-DQN, and A3C use matched custom PyTorch implementations. No per-dimension hyperparameter search is performed, so any performance difference across dimensions is attributable to the observation vector rather than to the training recipe.

**Rule-based baselines** span five management styles. The static-threshold

Table 1: The six evaluation scenarios and their primary stress mechanism.

| Scenario                           | Stress mechanism                                               |
|------------------------------------|----------------------------------------------------------------|
| <code>base_market</code>           | Stable, the cleanest test of policy signal exploitation        |
| <code>drift</code>                 | Gradual new-client score deterioration and rising defaults     |
| <code>adverse_stress</code>        | Abrupt shock: higher defaults, larger loans, capital pressure  |
| <code>class_imbalance_shift</code> | Growing share of weaker new applicants, tests composition risk |
| <code>noise</code>                 | Stable mean with noisy weekly volumes and measurements         |
| <code>split_policy_dynamics</code> | New clients deteriorate while repeat clients improve           |

baseline (`static_threshold`) applies one fixed shared threshold. The split-static baseline (`split_policy_static`) maintains separate fixed defaults per segment. The profit-oriented policy (`profit_oriented`) performs a weekly grid search maximising current-cohort expected profit without penalty terms. The risk-aware weekly policy (`risk_aware_weekly`) adjusts thresholds reactively to rolling default pressure and capital signals. The constraint-aware weekly policy (`constraint_aware_weekly`) runs a penalty-weighted weekly grid search that explicitly internalises default-rate, approval-rate, capital-usage, and volatility violations.

### 3.4. Reward Function

With delayed reward enabled, the shaped weekly reward is:

$$\begin{aligned}
r_t = & w_p \Pi_t^{\text{realized}} + w_n \text{NPV}_t^{\text{realized}} - w_s \hat{d}_t \cdot 100 \\
& - \lambda_d [\max(0, \bar{d}_t - 0.12)]^2 \cdot 1000 - \lambda_a |\alpha_t - 0.42| \cdot 1000 \\
& - \lambda_c [\max(0, c_t - 0.82)]^2 \cdot 1000 - \lambda_v \max\left(0, \frac{\sigma_{\Pi,t} - 8500}{8500}\right) \cdot 1000,
\end{aligned} \tag{1}$$

where  $\Pi_t^{\text{realized}}$  and  $\text{NPV}_t^{\text{realized}}$  are realized weekly profit and NPV (not current-cohort expectations),  $\hat{d}_t$  is the expected default rate of the current cohort,  $\bar{d}_t$  is the rolling realized default rate,  $\alpha_t$  is the weekly approval rate,  $c_t$  is the projected capital usage ratio, and  $\sigma_{\Pi,t}$  is realized profit volatility. The four constants are calibration targets: the default-rate cap (0.12), the target approval rate (0.42), the capital-usage cap (0.82), and the profit-volatility scale (8500). Active weights:  $w_p=1.0$ ,  $w_n=0.25$ ,  $w_s=0.35$ ,  $\lambda_d=9.0$ ,  $\lambda_a=2.0$ ,  $\lambda_c=3.5$ ,  $\lambda_v=1.4$ .

The reward is multi-objective by design. Rankings by cumulative reward need not coincide with rankings by raw profit, because the reward encodes portfolio risk constraints rather than profit alone. Throughout this paper, “realized profit” is computed from realized interactive weekly profits plus terminal settlement, that is, realized final portfolio profit rather than a current-cohort expectation.

## 4. Experimental Design

The experiment is fully controlled: *only* observation dimensionality varies across the four runs. Everything else (the simulator, the reward, the eight random seeds, the 50 training episodes per seed, the evaluation protocol,

the scenarios, the threshold grid, and the controller set) is held fixed. The full-profile scale used to generate the main summary tables is  $6 \text{ agents} \times 4 \text{ dimensions} \times 6 \text{ scenarios} \times 8 \text{ seeds} = 1,152$  training runs, with 576 evaluation runs per controller per dimension. A validity layer programmatically verifies that the first 12 observation features are bit-for-bit identical across all four state sizes on a common reset and three fixed-action transitions.

In Table 2, the cumulative composition of the four observation layers is shown, where each layer adds features that a portfolio manager would realistically observe at weekly decision time with no access to future outcomes. Because each layer is a superset of the previous one, observation dimensionality is the only factor that changes across the four conditions.

#### *4.1. Hypotheses*

The three hypotheses follow from the research gap and from the literature reviewed in Section 2, and each is tested on paired run-level data in Section 5.4. The first hypothesis concerns state representation. Because feature enhancement and richer inputs are repeatedly reported to help credit models up to a point (Yang et al., 2024), we expect added observation features to improve the RL frontier but only over an intermediate range. The second hypothesis concerns the RL advantage. The credit-scoring benchmarking literature repeatedly finds that more complex models do not reliably dominate simpler ones (Lessmann et al., 2015; Gunnarsson et al., 2021), and the direct precursor compared RL against only a single static baseline (Herasymovych et al., 2019; Kiatsupaibul et al., 2024), so we expect the universal-dominance claim to fail against strong rule-based policies. The third hypothesis concerns delayed evaluation. Because outcomes are delayed and terminal settlement is

Table 2: Cumulative observation layers. The first 12 features are invariant across all configurations and are verified programmatically before each experiment.

| Size | Added information                                                                                                                                                                                                                                                                             |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 12D  | Compact dashboard: week progress, overall/segment approval rates, rolling realized default rate, expected default rate, realized profit, profit volatility, projected capital usage, outstanding ratio, both threshold levels.                                                                |
| 20D  | 12D + current economic context: repeat segment share, expected profit/NPV per application, realized NPV, weekly reward, threshold gap, capital headroom, realized-vs-expected default gap.                                                                                                    |
| 30D  | 20D + lagged/rolling memory: second-lag approval/default/profit/capital, four-week rolling means and std of approval, default, and profit, lagged threshold changes.                                                                                                                          |
| 50D  | 30D + segment-specific signals and stability descriptors: segment approval/ default/profit indicators, accepted-share ratios, rolling reward moments, cumulative reward/profit to date, capital-usage volatility, outstanding-ratio changes, threshold-gap history, application-volume ratio. |

applied, as in the delayed-reward credit-control setting of Herasymovych et al. (2019), early-period performance may misrepresent final portfolio quality.

### **H1 (State representation).**

Richer state representations improve the best RL frontier from 12D to 30D, while the move from 30D to 50D does not necessarily create additional improvement.

## H2 (RL vs. rule-based).

RL improves over rule-based policies only in some scenarios and metrics, and the universal claim that RL robustly dominates strong rule-based baselines is expected to be *rejected*.

## H3 (Delayed evaluation).

Early-period performance is an unreliable proxy for final realized portfolio quality when outcomes are delayed and terminal settlement is applied. The “locally worse, globally better” pattern is expected to hold.

## 5. Results

### 5.1. Best Overall and Best RL Controller by Dimension

In Table 3, the strongest controller is reported when baselines and RL agents are pooled under the repository’s official ranking rule (highest cumulative reward, then highest profit, then lowest default rate), together with the strongest RL controller in each dimension (full-profile run, 8 seeds  $\times$  50 episodes per seed, with data and configuration in the public repository). The best RL profit peaks at 12D and then declines, so added state dimensions do not buy profit for the best RL agent.

By cumulative reward, the primary ranking metric, the constraint-aware weekly policy is the best overall controller in every dimension (reward 3,666,052). On raw realized profit it is statistically tied with the profit-oriented policy, which is marginally higher (3,803,168 vs. 3,802,547). The two differ by less than 0.02% and the difference is not economically meaningful. The flat baseline row confirms that the experiment isolates the effect of observation size on RL quality, because the leading rule-based policy is

Table 3: Best overall controller and best RL controller by state dimension (full-profile run). The baseline line is flat because the leading baseline does not depend on the RL observation vector. “Profit” is realized final portfolio profit (Section 3.4); “Reward” is cumulative shaped reward.

| Dim                                                | Controller              | Type     | Profit    | Reward    | Default rate |
|----------------------------------------------------|-------------------------|----------|-----------|-----------|--------------|
| <i>Best overall (pooled, by cumulative reward)</i> |                         |          |           |           |              |
| 12                                                 | Constraint-aware weekly | baseline | 3,802,547 | 3,666,052 | 0.0760       |
| 20                                                 | Constraint-aware weekly | baseline | 3,802,547 | 3,666,052 | 0.0760       |
| 30                                                 | Constraint-aware weekly | baseline | 3,802,547 | 3,666,052 | 0.0760       |
| 50                                                 | Constraint-aware weekly | baseline | 3,802,547 | 3,666,052 | 0.0760       |
| <i>Best RL agent</i>                               |                         |          |           |           |              |
| 12                                                 | Double-DQN              | RL       | 3,446,564 | 3,449,745 | 0.0756       |
| 20                                                 | Double-DQN              | RL       | 3,124,322 | 3,281,011 | 0.0694       |
| 30                                                 | Double-DQN              | RL       | 3,172,605 | 3,311,519 | 0.0725       |
| 50                                                 | Double-DQN              | RL       | 3,034,292 | 3,196,350 | 0.0700       |

independent of the RL state vector by construction. The best RL profit peaks at 12D and then declines, a saturation-and-regression pattern rather than a monotone improvement, with the 12D to 20D drop of  $-322,241$  being the sharpest. Default rate for Double-DQN improves from 7.56% at 12D to 6.94% at 20D, rises to 7.25% at 30D, and settles at 7.00% at 50D, which suggests that the risk-side benefit of additional features has a different shape than the profit benefit. At 12D, the gap between the best RL and the best overall controller is  $-355,983$  ( $-9.4\%$  in profit), a meaningful but not catastrophic shortfall.

Figure 3 shows the same comparison as a function of observation dimen-

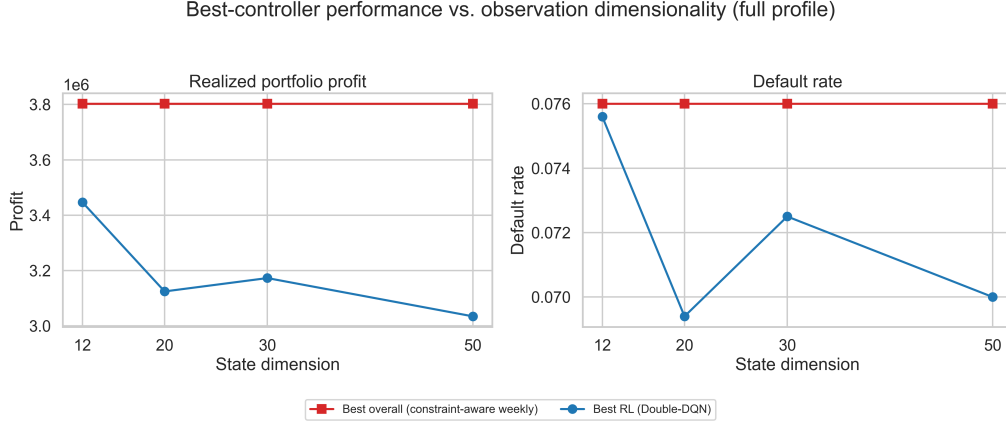


Figure 3: Best-controller performance versus observation dimensionality (full-profile run, 8 seeds). *Left*: realized portfolio profit. *Right*: default rate. The best overall controller (the constraint-aware weekly policy) is independent of the RL state vector and is therefore flat across dimensions, while the best RL agent (Double-DQN) peaks at 12D on profit and then declines, a saturation-and-regression pattern rather than monotone improvement.

sionality. The figure shows that added state dimensions do not improve profit for the best RL agent and in fact reverse it beyond 12D, while the leading baseline is unaffected by the state vector.

## 5.2. Agent Comparison

In Table 4, the mean performance of each agent across all four dimensions is summarised alongside the three principal rule-based baselines (full-profile run). Double-DQN leads all RL agents on profit, yet still falls 9.4% short of the best baseline at its 12D peak.

Double-DQN leads all RL agents on profit (3,194,446) and closely matches the top baselines’ approval rate (76.7%), but still falls 9.4% short of the best baseline at its 12D peak. PPO stands out for its low default rate (6.21%) despite moderate profit, and is numerically identical across 12D, 20D, 30D,



Table 4: Agent and baseline summary (mean across all four dimensions, full-profile run). A3C and A2C are reported in a single combined row. “Approval” denotes mean weekly approval rate.

| Controller              | Type     | Mean profit | Default rate | Approval |
|-------------------------|----------|-------------|--------------|----------|
| Constraint-aware weekly | baseline | 3,802,547   | 7.60%        | †        |
| Profit-oriented         | baseline | 3,803,168   | †            | †        |
| Static-threshold        | baseline | †           | †            | †        |
| Double-DQN              | RL       | 3,194,446   | 7.19%        | 0.767    |
| DQN                     | RL       | 2,960,000   | 7.02%        | 0.700    |
| PPO                     | RL       | 2,620,000   | 6.21%        | 0.572    |
| SAC                     | RL       | 1,820,000   | 6.63%        | 0.449    |
| A3C / A2C (avg)         | RL       | 2,470,000   | 6.60%        | 0.570    |

† Per-controller full-profile default- and approval-rate aggregates for the rule-based baselines (and the static-threshold profit) are not included in the released full-profile summary and are omitted rather than estimated. RL per-agent profit means are shown at the precision of the released aggregates. The Double-DQN mean is computed exactly from its four per-dimension values.

and 50D. Under the modest training budget PPO converges to a degenerate, essentially constant-threshold policy (consistent with the near-zero threshold volatility reported in Section 6.4), so the additional observation features are effectively unused rather than informative. SAC is the weakest RL agent overall and deteriorates further at 50D.

### 5.3. Scenario-Level Winners

In Table 5, the reward-based, profit-based, and lowest-default-rate winner for each scenario at the 50D configuration is reported. No single metric

Table 5: Scenario-level winners at 50D state (full-profile run, with the same qualitative pattern at 12D/20D/30D, shown here for 50D). *Reward winner*: highest cumulative shaped reward. *Profit winner*: highest realized profit. *Default winner*: lowest default rate. No RL agent wins any scenario by reward or profit. The risk-aware weekly policy is the single lowest-default controller in every scenario, and the best RL agent nonetheless undercuts both profit-seeking baselines (the profit-oriented and constraint-aware weekly policies) on default rate in a majority of scenarios.

| Scenario                           | Reward winner           | Profit winner           | Default winner    |
|------------------------------------|-------------------------|-------------------------|-------------------|
| <code>adverse_stress</code>        | Static-threshold        | Constraint-aware weekly | Risk-aware weekly |
| <code>base_market</code>           | Profit-oriented         | Profit-oriented         | Risk-aware weekly |
| <code>class_imbalance_shift</code> | Constraint-aware weekly | Profit-oriented         | Risk-aware weekly |
| <code>drift</code>                 | Profit-oriented         | Profit-oriented         | Risk-aware weekly |
| <code>noise</code>                 | Profit-oriented         | Profit-oriented         | Risk-aware weekly |
| <code>split_policy_dynamics</code> | Constraint-aware weekly | Profit-oriented         | Risk-aware weekly |

identifies one controller as best, because the reward, profit, and default-rate winners differ across scenarios.

RL wins zero of six scenarios by reward and zero by profit across all four dimensions. The profit-oriented policy wins raw profit in five of six scenarios, which is why the headline “best overall” ranking, based on cumulative reward, and the profit ranking need not coincide. However, the best RL agents undercut both profit-seeking baselines (the profit-oriented and constraint-aware weekly policies) on default rate in a majority of scenarios, while the risk-aware weekly policy remains the single lowest-default controller (Table 5). This result has direct practical significance: a portfolio manager willing to accept a small profit reduction in exchange for capital conservation may prefer

the richer-state RL policy over the profit-seeking baselines.

#### 5.4. Statistical Hypothesis Tests

In Table 6, the paired statistical comparisons for the three hypotheses of Section 4.1 are summarised (H1 and H2 on the full-profile run, H3 on the development profile, with the paired run-level data in the public repository). The tests confirm that 30D significantly beats 50D but not 12D, that RL is significantly weaker than the profit-oriented baseline, and that the RL deficit under stress does not reverse by episode end. The dimensionality hypotheses are tested on vanilla **DQN**, used here as the canonical discrete-action reference agent for the controlled state-size comparison. Double-DQN is reported separately as the best-performing RL agent, so DQN is the appropriate reference agent for the paired tests.

**H1** (richer state improves RL up to an intermediate representation) is *partially supported*. The 30D to 12D reward difference is positive but insignificant ( $p = 0.41$ ), so a 30D advantage over 12D is not established. The 30D to 50D comparison is highly significant ( $p < 10^{-3}$ , mean +380,188), which confirms that 50D reliably underperforms 30D. In short, 30D is significantly better than 50D, but not significantly better than 12D.

**H2** (RL does not universally dominate) is strongly confirmed. Both the 30D and 50D DQN agents are significantly weaker than the profit-oriented policy by many hundreds of thousands of reward units ( $p < 10^{-3}$  in both comparisons).

**H3** (delayed evaluation changes rankings, “locally worse globally better”) is *not supported* in the adverse-stress setting. Because intra-episode weekly trajectory data is only available for the development (quick) profile, a 26-week

Table 6: Paired statistical checks. The H1 and H2 rows are reported for the full-profile run. The H3 rows ( $\dagger$ ) are computed on the development (quick) profile, a 26-week interactive horizon, which is the only profile for which intra-episode weekly trajectory data is available in the repository. Week 25 is the *final* week of that 26-week horizon, not a mid-point of the 52-week full run. *Mean diff.*: left minus right controller, where positive values favour the left controller on reward/profit and negative values favour it on default rate. CI: 95% percentile bootstrap.

| Hyp.         | Paired comparison                          | Metric | Mean diff. | 95% CI               | $p$        |
|--------------|--------------------------------------------|--------|------------|----------------------|------------|
| H1           | 30D DQN – 12D DQN                          | Reward | +13,021    | [−17,654, +44,263]   | 0.412      |
| H1           | 30D DQN – 50D DQN                          | Reward | +380,188   | [+355,177, +404,236] | $<10^{-3}$ |
| H2           | 30D DQN – Profit-oriented                  | Reward | −368,145   | [−384,214, −351,974] | $<10^{-3}$ |
| H2           | 50D DQN – Profit-oriented                  | Reward | −748,333   | [−773,453, −722,900] | $<10^{-3}$ |
| H3 $\dagger$ | DQN – Static-threshold<br>(week 6, early)  | Profit | −1,484     | [−4,533, +1,519]     | 0.386      |
| H3 $\dagger$ | DQN – Static-threshold<br>(week 25, final) | Profit | −69,016    | [−149,906, +9,403]   | 0.131      |

interactive horizon, the H3 checkpoints are computed on that profile and labelled accordingly in Table 6. The DQN agent trails the static-threshold baseline both at the early checkpoint (week 6, profit gap −1,484) and at week 25, the final week of the 26-week development horizon (profit gap −69,016). The deficit therefore does not reverse by episode end, the opposite of the “locally worse, globally better” pattern, so terminal settlement reveals a *persistent* deficit for the RL agent under stress rather than a recovery trajectory. With only twelve paired runs, however, neither gap is statistically significant ( $p = 0.39$  and  $p = 0.13$ , respectively), so the persistence of the

deficit is directional rather than statistically established under this profile.

## 6. Discussion

### 6.1. *Why Rule-Based Baselines Win*

The leading baselines (the constraint-aware weekly and profit-oriented policies) perform a weekly grid search over the same 121 joint threshold actions as the RL agents. Their advantage is structural: they evaluate each action directly on the current week’s cohort economics, they see the realized profit and default signals immediately, and they set thresholds according to an explicit objective. Under a 50-episode training budget, RL agents have not seen enough portfolio trajectories to learn a value function that is competitive with this direct weekly evaluation. The gap would likely narrow with longer training, but the current results establish a robust finding: when a well-designed rule-based policy has access to the same action grid and the same current-week signal, it outperforms RL under short budgets.

### 6.2. *The Conditional Value of RL*

Despite losing on headline profit and reward, RL agents achieve lower default rates than the two profit-seeking baselines in a majority of scenarios, though the risk-aware weekly policy remains the lowest-default controller overall (Table 5). A portfolio manager who values capital conservation may accept a modest profit reduction in exchange for this risk-side improvement. This portfolio-quality advantage is not visible in the headline reward metric, and it is the main condition under which the richer-state RL policies add value.

### 6.3. Dimensionality Saturation

The 30D state adds lagged and rolling history (second-lag approval, default, and profit values, four-week rolling means and standard deviations, and lagged threshold changes). This temporal context is what a controller needs under slow-moving drift or noisy volume signals, namely the ability to compare this week with the recent past. The 50D state adds richer segmentation and stability descriptors, but those extra features require more optimization work and offer diminishing marginal information under the current training budget. The result is that 30D is the most balanced point on the *risk-and-information* axis: it delivers more temporal memory than 12D while avoiding the regression that 50D introduces for most algorithms. On raw profit, however, 12D is not significantly worse (Section 5.4), so the case for 30D rests on its risk-side and temporal-memory benefits rather than on a profit advantage.

### 6.4. Episode-Static RL Behavior

A notable finding is that the best RL policies choose one threshold pair at the beginning of the episode and largely maintain it throughout the interactive horizon. The threshold volatility of the top RL winners is 0.00 at 12D, 20D, and 30D (and 0.44 at 50D), which confirms near-static within-episode behavior. The dimensionality improvement therefore manifests not as richer weekly adaptation but as better initial threshold selection, because the agent learns to identify a better static split policy for each episode type. This contrasts with the leading baselines, whose weekly grid searches produce substantial threshold volatility. The implication for practitioners is that richer state may first improve episode-level threshold selection before enabling genuine week-to-week adaptation, and that validating true dynamic adjustment requires

longer training horizons.

### 6.5. *Practical Recommendations*

For a lending business the results translate into concrete guidance.

- When the training budget is short, deploy the weekly grid-search controller (the constraint-aware or profit-oriented weekly policy). It matches or beats every RL agent on profit and reward and needs no training.
- Adopt an RL controller only when the institution values default-rate reduction and can accept a small profit trade-off, because the richer-state RL policies undercut the profit-seeking baselines on default rate in most scenarios.
- Use a 30D state as a sensible default for the risk-side and temporal-memory benefits, but expect no profit gain over 12D, and avoid 50D, which reliably underperforms 30D at the current training budget.

### 6.6. *Limitations*

First, all results derive from a synthetic simulator, so calibration to any specific production portfolio would shift the absolute numbers and potentially alter some rankings. Second, the 50-episode-per-seed training budget is modest in order to isolate the state-dimensionality signal, so convergence claims would require substantially more training. Third, controller rankings are sensitive to the reward weights in Equation (1), and a reward sensitivity analysis is left for future work. Fourth, bootstrap confidence intervals quantify simulation uncertainty, not real-world external validity.

These results are bounded by the synthetic setting. The default and recovery parameters are calibrated to aggregate statistics rather than fitted

to a single proprietary book, so the absolute profit and default levels should be read as order-of-magnitude rather than exact. The training budget is deliberately short, which favours strong rule-based search and limits what the reinforcement-learning agents can recover, so a larger budget could narrow the gap reported here. The real-data comparison validates the score-to-default relationship and the parameter ranges, but it does not validate the weekly threshold dynamics directly, which would require a lender to deploy and log a controlled threshold policy.

## 7. Threats to validity

The conclusions of this study rest on a synthetic simulator, so their external validity depends on how faithfully the simulator reproduces the dynamics of real unsecured consumer lending. This section states the main threats, compares the simulator against published real-world evidence, and explains why an agent-based simulator remains the appropriate tool under these threats.

### *7.1. Construct validity and simulator fidelity*

The portfolio is generated from parametric assumptions rather than observed loans, which introduces three concrete risks. First, the logistic default model in Appendix A imposes a smooth monotone link between score and default probability, whereas real default curves can be locally non-monotone in thin score regions. Second, the weekly event schedule collapses repayment, default, and recovery into a small set of timed cashflows, which understates the heterogeneity of real repayment paths. Third, the scenario set spans six stylised regimes, so it cannot represent every macro-financial path a real book



would face. We mitigate these risks by calibrating every primary parameter to published statistics and by comparing the score-to-default relationship against a large real loan book, as described next.

### *7.2. Comparison with published evidence and a real loan book*

Real consumer credit portfolios show a wide spread of default rates and strong class imbalance (Brown & Mues, 2012), which the scenario set is designed to span. Table 7 places each primary simulator parameter next to the corresponding real-world range and its source. In Table 7, the portfolio default rate, the loan tenor, the effective interest, and the recovery assumption all fall inside the ranges reported for unsecured retail lending, and the acceptance-threshold mechanism reproduces the setup of Herasymovych et al. (2019), whose optimal acceptance threshold on a real online consumer-loan book sits in the same mid-score region used here. Figure 4 overlays the simulator default-probability curves on the empirical default rate by within-book credit-score percentile estimated from the Lending Club loan book (Xia et al., 2017). In Figure 4, plotting against the within-book percentile aligns the two score domains, and both relationships fall monotonically while the higher Lending Club levels reflect its higher-rate and longer-tenor borrower mix.

Table 7: Calibration of the simulator against published evidence. In Table 7, each primary parameter sits inside a real-world range, and the sources are aggregate statistics and peer-reviewed studies rather than a single proprietary book.

| Parameter              | Simulator                          | Real-world range (source)                                                                                                                                                                     |
|------------------------|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Portfolio default rate | 4–14% across scenarios             | around 4–5% in normal conditions and up to 12.9% under stress for unsecured consumer loans (Bank of Russia, 2025)<br>20.0% on Lending Club resolved loans (own computation; Xia et al., 2017) |
| Acceptance mechanism   | weekly score threshold, grid 35–85 | optimal threshold in the mid-score region on a real online book (Herasymovych et al., 2019)                                                                                                   |
| Loan tenor             | 4–12 months                        | short-term consumer and online installment products (Herasymovych et al., 2019)<br>Lending Club is longer, with 76% at 36 months and 24% at 60 months                                         |
| Loan amount            | 0.3–9.0 k currency units           | small-ticket loans typical of short-term online consumer lending (Herasymovych et al., 2019)<br>Lending Club median is larger, at 12,000 USD                                                  |
| Effective interest     | 11–15% annualised                  | comparable to the Lending Club median of 12.7%                                                                                                                                                |
| Recovery on default    | 26–34% (LGD $\approx$ 66–74%)      | recovery distributions for unsecured consumer loans (Loterman et al., 2012; Qi & Zhao, 2011)                                                                                                  |

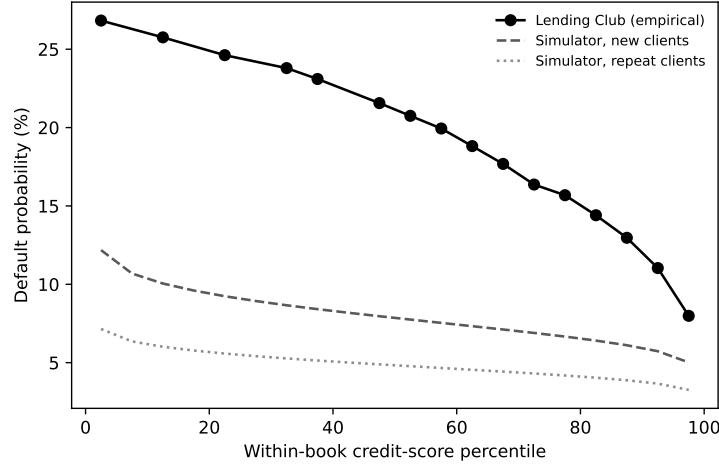


Figure 4: Default probability by within-book credit-score percentile, simulator curves versus the Lending Club loan book. In Figure 4, the within-book percentile axis aligns the two score domains, and the higher Lending Club levels reflect its higher-rate and longer-tenor borrower mix.

### 7.3. Why an agent-based simulator

Loan-level data that jointly record the score, the funding decision, the realised outcome, and the calendar timing across several market regimes are rarely public, because lender underwriting models and outcome histories are proprietary assets (Di Maggio et al., 2024). In the absence of such data, an agent-based simulator acts as a controlled laboratory that varies one factor at a time, here the observation dimension, while holding the protocol fixed, which is the comparison that observational data cannot deliver (Axtell & Farmer, 2025). We therefore read the simulator as a tool for studying the structure of the control problem under a fixed training budget, not as a forecast of any single lender’s profit.

## 8. Conclusion

This paper evaluates six reinforcement learning algorithms against five rule-based baselines across six market scenarios and four observation-space sizes (12D, 20D, 30D, 50D) in a controlled credit portfolio simulation. The main empirical findings are:

- (1) The constraint-aware weekly policy is the best overall controller in every dimension by cumulative reward (reward 3,666,052). On raw realized profit it is statistically tied with the profit-oriented policy (3,802,547 vs. 3,803,168, the latter marginally higher). RL wins zero of six scenarios by reward or profit across all dimensions.
- (2) Double-DQN is the best RL agent, peaking at 12D (profit 3,446,564, reward 3,449,745) and declining at higher dimensionality. The 12D RL-to-baseline gap is  $-355,983$  ( $-9.4\%$ ).
- (3) The 30D to 12D reward improvement ( $+13,021$ ) is not statistically significant ( $p = 0.41$ ), but 30D significantly outperforms 50D ( $+380,188$ ,  $p < 10^{-3}$ ), which confirms that 50D dimensions reliably harm rather than help at the current budget.
- (4) RL achieves lower default rates than the two profit-seeking baselines in a majority of scenarios, though the risk-aware weekly policy stays the lowest-default controller overall, which demonstrates a conditional risk-side advantage.
- (5) The best RL policies are episode-static, so richer observation improves threshold *selection*, not within-episode adaptation.

The practical implication is direct: when training budgets are short and well-designed rule-based policies have access to the same threshold grid,

strong baselines remain the benchmark to beat. The 30D observation space is a reasonable default for credit-portfolio RL because it adds economically meaningful lagged and rolling memory and supports the conditional risk-side advantage. We do *not* claim a 30D profit advantage, because on profit and reward 12D is not significantly worse. Future work should investigate longer training horizons to test whether the RL-to-baseline gap closes, reward sensitivity to penalty weights, more adaptive recurrent or attention-based architectures, and validation on real or semi-synthetic loan panels.

## References

- Arulkumaran, K., Deisenroth, M. P., Brundage, M., & Bharath, A. A. (2017). Deep reinforcement learning: A brief survey. *IEEE Signal Processing Magazine*, 34(6), 26–38. <https://doi.org/10.1109/MSP.2017.2743240>
- Axtell, R. L., & Farmer, J. D. (2025). Agent-based modeling in economics and finance: Past, present, and future. *Journal of Economic Literature*. <https://doi.org/10.1257/jel.20221319>
- Baesens, B., Van Gestel, T., Viaene, S., Stepanova, M., Suykens, J., & Vanthienen, J. (2003). Benchmarking state-of-the-art classification algorithms for credit scoring. *Journal of the Operational Research Society*, 54(6), 627–635. <https://doi.org/10.1057/palgrave.jors.2601545>
- Bank of Russia. (2025). *Financial stability review*. Bank of Russia. <https://www.cbr.ru/eng/analytics/finstab/ofs/>
- Bellotti, T., & Crook, J. (2009). Support vector machines for credit scoring

- and discovery of significant features. *Expert Systems with Applications*, 36(2), 3302–3308. <https://doi.org/10.1016/j.eswa.2008.01.005>
- Brown, I., & Mues, C. (2012). An experimental comparison of classification algorithms for imbalanced credit scoring data sets. *Expert Systems with Applications*, 39(3), 3446–3453. <https://doi.org/10.1016/j.eswa.2011.09.033>
- Dayan, P., & Niv, Y. (2008). Reinforcement learning: The good, the bad and the ugly. *Current Opinion in Neurobiology*, 18(2), 185–196. <https://doi.org/10.1016/j.conb.2008.08.003>
- Di Maggio, M., Ratnadiwakara, D., & Carmichael, D. (2024). Invisible primes: Fintech lending with alternative data. *Management Science*. <https://doi.org/10.1287/mnsc.2024.07854>
- Gunnarsson, B. R., vanden Broucke, S., Baesens, B., Óskarsdóttir, M., & Lemahieu, W. (2021). Deep learning for credit scoring: Do or don't? *European Journal of Operational Research*, 295(1), 292–305. <https://doi.org/10.1016/j.ejor.2021.03.006>
- Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning* (PMLR 80, pp. 1861–1870). <https://proceedings.mlr.press/v80/haarnoja18b.html>
- Hand, D. J., & Henley, W. E. (1997). Statistical classification methods in consumer credit scoring: A review. *Journal of the Royal Statistical Soci-*

- ety: Series A*, 160(3), 523–541. <https://doi.org/10.1111/j.1467-985X.1997.00078.x>
- Herasymovych, M., Märka, K., & Lukason, O. (2019). Using reinforcement learning to optimize the acceptance threshold of a credit scoring model. *Applied Soft Computing*, 84, 105697. <https://doi.org/10.1016/j.asoc.2019.105697>
- Kiatsupaibul, S., Chansiripas, P., Manopanjaisiri, P., Visantavarakul, K., & Wen, Z. (2024). *Reinforcement learning in credit scoring and underwriting* [Preprint]. arXiv. <https://doi.org/10.48550/arXiv.2212.07632>
- Lessmann, S., Baesens, B., Seow, H.-V., & Thomas, L. C. (2015). Benchmarking state-of-the-art classification algorithms for credit scoring: An update of research. *European Journal of Operational Research*, 247(1), 124–136. <https://doi.org/10.1016/j.ejor.2015.05.030>
- Loterman, G., Brown, I., Martens, D., Mues, C., & Baesens, B. (2012). Benchmarking regression algorithms for loss given default modeling. *International Journal of Forecasting*, 28(1), 161–170. <https://doi.org/10.1016/j.ijforecast.2011.01.006>
- Nallakaruppan, M. K., Balusamy, B., Shri, M. L., Malathi, V., & Bhat-tacharyya, S. (2024). An explainable AI framework for credit evaluation and analysis. *Applied Soft Computing*, 153, 111307. <https://doi.org/10.1016/j.asoc.2024.111307>
- Qi, M., & Zhao, X. (2011). Comparison of modeling methods for loss given

- default. *Journal of Banking & Finance*, 35(11), 2842–2855. <https://doi.org/10.1016/j.jbankfin.2011.03.011>
- Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., & Dormann, N. (2021). Stable-Baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268), 1–8. <http://jmlr.org/papers/v22/20-1364.html>
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). *Proximal policy optimization algorithms* [Preprint]. arXiv. <https://doi.org/10.48550/arXiv.1707.06347>
- Siddiqi, N. (2006). *Credit risk scorecards: Developing and implementing intelligent credit scoring*. Wiley.
- Thomas, L. C. (2017). *Credit scoring and its applications* (2nd ed.). Society for Industrial and Applied Mathematics. <https://doi.org/10.1137/1.9781611974560>
- van Hasselt, H., Guez, A., & Silver, D. (2016). Deep reinforcement learning with double Q-learning. In *Proceedings of the 30th AAAI Conference on Artificial Intelligence* (Vol. 30, pp. 2094–2100). AAAI Press. <https://doi.org/10.1609/aaai.v30i1.10295>
- Xia, Y., Liu, C., Li, Y., & Liu, N. (2017). A boosted decision tree approach using Bayesian hyper-parameter optimization for credit scoring. *Expert Systems with Applications*, 78, 225–241. <https://doi.org/10.1016/j.eswa.2017.02.017>



Yang, D., Xiao, B., Cao, M., & Shen, H. (2024). A new hybrid credit scoring ensemble model with feature enhancement and soft voting weight optimization. *Expert Systems with Applications*, 238, 122101. <https://doi.org/10.1016/j.eswa.2023.122101>

Zhang, X., & Yu, L. (2023). Consumer credit risk assessment: A review from the state-of-the-art classification algorithms, data traits, and learning methods. *Expert Systems with Applications*, 237, 121484. <https://doi.org/10.1016/j.eswa.2023.121484>

## Appendix A. Simulator details

This appendix specifies the data-generating process so that the experiments can be reproduced from the public repository. All values are the calibrated full-profile settings.

*Applicant generation..* Each interactive week the simulator draws a base application volume around 600 applications, scales it by a seasonal factor and multiplicative volume noise, and splits it into new and repeat counts through two Poisson draws. The repeat share follows a scenario base level modulated by an annual seasonal term and clamped to  $[0.15, 0.75]$ . Each applicant in segment  $s$  receives a credit score drawn from a Gaussian  $\mathcal{N}(\mu_s, \sigma_s)$  clipped to  $[0, 100]$ , with  $\mu_{\text{new}} = 59$ ,  $\sigma_{\text{new}} = 13$  and  $\mu_{\text{repeat}} = 69$ ,  $\sigma_{\text{repeat}} = 10$  in the base market.

*Default and recovery..* An applicant with score  $x$  in segment  $s$  defaults with probability

$$p_s(x) = \sigma\left(\beta_s - \kappa_s \cdot \frac{x-50}{10}\right), \quad (\text{A.1})$$

clipped to  $[0.01, 0.98]$ , where  $\sigma$  is the logistic function and  $\beta_{\text{new}} = -2.3$ ,  $\kappa_{\text{new}} = 0.18$ ,  $\beta_{\text{repeat}} = -2.6$ ,  $\kappa_{\text{repeat}} = 0.20$  in the base market. Scenario stress shifts  $\beta_s$  gradually over the horizon and an abrupt shock adds to  $\beta_s$  from the shock week onward (Table A.8). These settings target portfolio default rates of 4–6% in the base market and 9–14% under adverse stress. A defaulted loan recovers with probability  $r_{\text{new}} = 0.26$  or  $r_{\text{repeat}} = 0.34$ , and on recovery the lender receives the principal plus a fraction  $\varphi_s$  of the scheduled interest, with  $\varphi_{\text{new}} = 0.65$  and  $\varphi_{\text{repeat}} = 0.72$ .

*Loan terms and cashflows.* The principal is lognormal with mean  $A_{\text{new}} = 1650$  or  $A_{\text{repeat}} = 2150$  currency units and log-scale 0.28 (new) or 0.25 (repeat), clipped to  $[300, 9000]$  and scaled by the scenario loan-amount multiplier. Each loan receives an integer term  $d$  in months (new: 5–12, repeat: 4–10) and a scheduled interest income of  $\text{principal} \times (\rho_s d/12 + o_s)$ , with annual fee rate  $\rho_{\text{new}} = 0.16$ ,  $\rho_{\text{repeat}} = 0.13$  and origination  $o_{\text{new}} = 0.035$ ,  $o_{\text{repeat}} = 0.03$ . Outcomes enter a forward event queue: a repaid loan returns the interest at week issue +  $d$ , a defaulted loan returns  $-\text{principal}$  at week issue +  $d$ , and a recovered loan adds the recovery value a further 1–4 weeks later. The term index  $d$  therefore sets both the interest horizon  $d/12$  years and the event delay  $d$  weeks. Each cashflow is discounted to net present value at a weekly rate of  $0.14/52$ , that is an annual discount rate of 14%.

*Episode structure and capital.* Each episode runs an 8-week warm-up at the default thresholds (60 for new, 50 for repeat), then a 52-week interactive horizon, then a terminal settlement that resolves all pending events so that loans originated near the horizon are not penalised. Outstanding principal

plus the principal accepted in the current week is tracked against a capital limit of 3,000,000 to form the capital-usage ratio used in the reward.

Table A.8: Per-scenario stress parameters. In Table A.8, the score and default shifts are total drifts applied gradually over the horizon, and the shock column gives the magnitude added to the default intercept and the week it begins.

| Scenario              | Repeat share | Score shift (new/rep) | Default shift (new/rep) | Score noise | Loan mult. | Shock (week) |
|-----------------------|--------------|-----------------------|-------------------------|-------------|------------|--------------|
| base_market           | 0.35         | 0.00 / 0.00           | 0.00 / 0.00             | 1.00        | 1.00       | none         |
| drift                 | 0.34         | -0.18 / -0.07         | 0.18 / 0.08             | 1.00        | 1.00       | 0.08 (13)    |
| adverse_stress        | 0.33         | -0.10 / -0.10         | 0.30 / 0.22             | 1.15        | 1.15       | 0.22 (10)    |
| class_imbalance_shift | 0.28         | -0.08 / 0.03          | 0.12 / -0.02            | 1.00        | 0.95       | 0.12 (16)    |
| noise                 | 0.35         | 0.00 / 0.00           | 0.04 / 0.04             | 1.45        | 1.00       | none         |
| split_policy_dynamics | 0.38         | -0.20 / 0.10          | 0.22 / -0.10            | 1.00        | 1.00       | 0.06 (14)    |

## Appendix B. Training hyperparameters

In Table B.9, the shared training configuration for the full-profile run is listed. The settings are held identical across all four state sizes, so any performance difference between dimensions is attributable to the observation vector rather than to the training recipe.

Table B.9: Shared training settings (full-profile run), held identical across all four state sizes. In Table B.9, these settings are fixed so that any difference across dimensions reflects the observation vector rather than the training recipe.

| Parameter                | Value                            | Notes                      |
|--------------------------|----------------------------------|----------------------------|
| Network architecture     | MLP, $2 \times 128$ hidden units | Shared across all agents   |
| Discount factor $\gamma$ | 0.98                             | Standard                   |
| Learning rate            | SB3 default (varies by agent)    | Not tuned                  |
| Training episodes/seed   | 50                               | Full profile               |
| Seeds                    | 11, 23, 37, 51, 67, 79, 97, 113  | 8 seeds, fixed across dims |
| Evaluation runs          | 12 per seed per scenario         | Same protocol across dims  |
| Threshold grid           | [35, 85] in steps of 5           | 11 actions per segment     |
| Warm-up weeks            | 8                                | Pre-fills loan book        |
| Interactive horizon      | 52 weeks                         | Full profile               |
| Applications per week    | 600                              | Before scale factors       |
| Bootstrap resamples      | 1000                             | 95% CI                     |

Table B.10: Per-agent reinforcement-learning hyperparameters (full-profile run), held identical across all four state sizes. In Table B.10, A2C, PPO, and SAC use Stable-Baselines3 (Raffin et al., 2021), while DQN, Double-DQN, and A3C use matched custom PyTorch implementations.

| Agent      | Implementation | Key hyperparameters                                                                                                                                                                 |
|------------|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DQN        | Custom         | lr $10^{-3}$ , batch 64, replay $2 \times 10^4$ , learning starts 300, train freq 1, gradient steps 1, target update 200, $\varepsilon$ $1.0 \rightarrow 0.05$ over 65% of training |
| Double-DQN | Custom         | As DQN, with a double-Q target                                                                                                                                                      |
| PPO        | SB3            | lr $3 \times 10^{-4}$ , rollout $n = 256$ , batch 64, GAE $\lambda$ 0.95, entropy coef 0.01, clip range 0.20                                                                        |
| A2C        | SB3            | lr $7 \times 10^{-4}$ , rollout $n = 256$ , GAE $\lambda$ 0.95, entropy coef 0.01                                                                                                   |
| A3C        | Custom         | lr $5 \times 10^{-4}$ , entropy coef 0.01, value coef 0.5, full-episode rollouts (single worker)                                                                                    |
| SAC        | SB3            | lr $3 \times 10^{-4}$ , buffer $5 \times 10^4$ , learning starts 500, batch 128, $\tau$ 0.02, train freq 1, gradient steps 1                                                        |

### CRedit authorship contribution statement

**Evgeny Baulin:** Software, Investigation, Writing – original draft. **P. P. Lukianchenko:** Supervision, Writing – review & editing.

**Declaration of competing interest**

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

**Acknowledgements**

This work was supported by the Ministry of Economic Development of the Russian Federation under Agreement 139-15-2025-011 (identifier 000000C313925P4G0002).